Week 12: Reliability and Final Project Launch

Replication, read/write choices, backup/restore thinking, and a realistic final project start

Reliability

What keeps a database usable when systems fail or data must be recovered?

Tradeoffs

What do read preference, write concern, replicas, and backups actually decide?

Project Start

Choose a small cloud database project and begin with evidence in mind.

What the Syllabus Says

Week 12: replication and reliability concepts, read preference, write concern, backup/restore

First Half of Class

Replication vocabulary
Primary and secondary roles
Automatic failover
Read preference and write concern
Backup and restore mindset

Second Half of Class

Final project overview
Platform choice: Supabase, Atlas, or both
Beginner-friendly project scenarios
Project artifacts and plan
Reliability evidence from the start

The week works when reliability concepts feed directly into final project decisions.

Reliability Is Not Magic

Managed cloud databases still require administrative reasoning

Availability

Can the database keep serving useful work when something fails?

Durability

After a write is accepted, how confident are we that it survives failures?

Recoverability

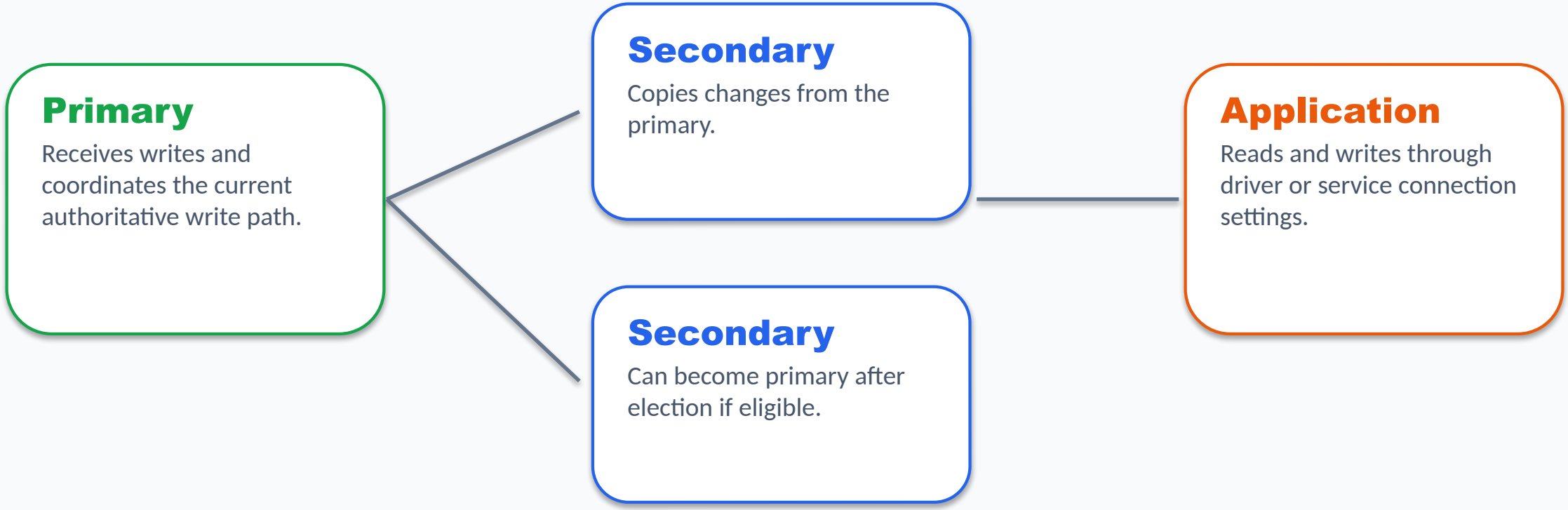
Can we restore usable data after a mistake or incident?

Course connection

Molina/Ullman frames durability through logging, recovery, and failure handling. Cloud platforms automate pieces of that work, but administrators still plan, verify, and document recovery.

Replica Set Mental Model

One primary accepts writes; secondaries copy changes and can help with resilience



Key idea: replication improves resilience, but replicas may lag behind the primary.

Read Preference

Where should read operations go?

primary

Default MongoDB behavior: read from the primary.

secondary

Read from secondary members; useful for some workloads but may be stale.

nearest

Prefer low-latency members based on topology and driver rules.

Plain-language tradeoff

Reading from replicas can reduce load or latency, but asynchronous replication means the answer may not reflect the newest primary state.

Write Concern

How much acknowledgement do we require before a write counts as accepted?

Lower concern

Can acknowledge faster, but may offer weaker durability across failures.

majority

Requires acknowledgement from a majority of voting replica set members.

Timeouts

A write can succeed locally but report concern errors if replication acknowledgement is delayed.

Write concern is a durability and acknowledgement choice, not just a performance setting.

Replication Lag and Failover

The two reliability behaviors students should be able to describe

Replication Lag

Delay between a write on the primary and application of that change on a secondary.
Small lag may be acceptable.
Large lag can affect stale reads and recovery behavior.

Failover

If the primary is unavailable, an election can choose a new primary.
Writes pause until election completes.
Applications should tolerate brief disruption.

Reliability means planning for delay and interruption, not pretending they never happen.

Backup Is Not Restore

A backup only matters if you can recover usable data from it

Backup

A stored copy, snapshot, dump, or recovery point.

Restore

The process of bringing data back into a usable system.

Verification

Checks proving the restored database actually works.

Restore-first mindset

Plan the restore before an incident. Know expected downtime, data loss risk, and the checks that prove recovery worked.

Atlas and Supabase Reliability Surfaces

Different platforms, similar administrative questions

MongoDB Atlas

Replica sets and automatic failover
Read preference and write concern
Atlas backup and restore features
Document model and index decisions

Supabase / Postgres

Managed Postgres project
Backups and PITR depending on plan
Read replicas on supported paid plans
SQL schema, indexes, roles, and RLS

The final project can use either platform, but every project needs an administrative story.

A Beginner Restore Runbook

What students should be able to write for the final project

1. Incident

What happened? Deleted rows, bad import, failed migration, or outage?

2. Restore Source

Which backup, snapshot, dump, or recovery point would be used?

3. Restore Steps

Where would restore happen and what downtime is expected?

4. Verification

Which queries prove the recovered database is usable?

A final project runbook can be short. It just needs to be concrete enough that another person could follow the plan.

Final Project Goal

Build a small cloud database project with administrative evidence

Not a giant app

The project is graded on database work, not on building a full production UI.

Database evidence

Model, seed data, queries, index choice, access-control concern, and reliability plan.

Portfolio value

A clear, small project is easier to explain in interviews than an unfinished large one.

Research-informed design choice

Beginner database projects work best when they solve a small, concrete domain and require visible database decisions instead of open-ended application sprawl.

Choose One Project Path

Postgres/Supabase, MongoDB/Atlas, or both

Path A: Supabase/Postgres

Best for clear tables, relationships, SQL queries, constraints, roles, RLS, and relational backup planning.

Path B: MongoDB/Atlas

Best for document models, embedded arrays, flexible records, MQL queries, Atlas indexes, and Atlas reliability planning.

Path C: Both

Use only when the split is simple: structured core records in Postgres and flexible events or logs in MongoDB.

Choose both only with a real reason. More tools does not automatically mean a better project.

Beginner-Friendly Project Scenarios

Small domains produce better final projects

Campus club event tracker

Keep scope small; focus on database evidence.

Small inventory tracker

Keep scope small; focus on database evidence.

Course resource library

Keep scope small; focus on database evidence.

Tutoring scheduler

Keep scope small; focus on database evidence.

Support ticket tracker

Keep scope small; focus on database evidence.

Book lending tracker

Keep scope small; focus on database evidence.

Budget or expense tracker

Keep scope small; focus on database evidence.

Student submission tracker

Keep scope small; focus on database evidence.

Scope rule

3-5 Postgres tables, or 2-4 MongoDB collections, or a very small split if using both platforms.

Minimum Project Review Evidence

What the instructor must be able to verify

Data Artifacts

- Schema or document model
- Seed data files
- Query examples
- Platform choice rationale

Cloud Review Access

- Atlas: add instructor to project/team
- Postgres: submit SQL/project files
- GitHub optional
- Access must allow review

Admin Evidence

- Index decision
- Access-control concern
- Backup/restore plan
- Restore verification checklist
- Reliability note

Bad Scope vs Good Scope

Make the project finishable

Too Big

A full e-commerce site, full LMS, full social network, complex multi-tenant SaaS app, or anything that depends on paid cloud features.

Good Scope

A small database that manages one real domain, includes enough records to test queries, and documents the admin choices clearly.

The final project should be boring enough to finish and real enough to explain.

Day 1 Lab: Reliability and Project Choice

Students leave with a path and a scenario

Reliability Notes

Define primary, secondary, replication lag, failover
Explain read preference and write concern
Name one reliability risk

Project Choice

Choose Supabase, Atlas, or both
Choose a small scenario
List core data
Name one backup or restore concern

In-class checkpoint: week12_day1_project_choice.md

Day 2 Lab: Restore-First Project Studio

Students start the final project plan

Restore-First Notes

Define backup, restore, PITR
Write a short verification checklist
Compare one Supabase and one Atlas reliability idea

Project Plan

Title and problem statement
Platform path
Data model plan
Seed data plan
Query plan
Admin and reliability plan

In-class checkpoints: week12_day2_reliability_notes.md and final_project_plan.md

End-of-Week Checklist

What students should have before leaving Week 12

Student Checklist

I chose Supabase/Postgres, MongoDB/Atlas, or both.
I chose a small project scenario.
I can explain one reliability concept in plain language.
I wrote a first data model plan.
I listed at least four project queries.
I wrote one backup/restore verification idea.
I know what to build first next week.